Zonoth

SESSION 01 · 2H

Containers & Docker: the mental model

What a container really is, how Docker runs one, and the handful of commands you will use for the next twenty hours.

Why containers | Namespaces & cgroups | Images & registries | Docker CLI | Lab 01

SESSION 01

What we'll cover today

1 · Why containers, and what one is

The "works on my machine" problem, VMs vs containers, and the Linux kernel features — namespaces, cgroups, overlayfs — that make a container.

2 · Images, registries, lifecycle

OCI, the Docker architecture (CLI → dockerd → containerd → runc), image naming and digests, and the states a container moves through.

3 · The everyday CLI

Three live demos plus the flags, `logs`, `exec`, `inspect`, `stats` and clean-up commands you will use daily.

4 · Lab 01

Run the Pinboard database tier: a real PostgreSQL container, a table, some rows — and the lesson that data dies with the container.

01

Why containers

The problem containers solve is not "servers are expensive" — it is that software refuses to behave the same way twice.

THE PROBLEM

"It works on my machine"

The same application behaves differently on every machine it touches.

- Different OS and library versions — glibc, OpenSSL, Python, the JDK
- A dependency someone installed by hand, two years ago
- Configuration living in a shell profile nobody wrote down
- Install instructions that rot the day after they are written

What we did before

Golden VM images, install scripts, configuration management. It works — but the artefact is gigabytes, takes minutes to boot, and every environment still drifts apart over time.

⚠ The unit we shipped was **instructions** for building an environment, not the environment itself.

COMPARE

Virtual machines vs. containers

	VIRTUAL MACHINE	CONTAINER
Isolation by	A hypervisor virtualising hardware	Kernel features: namespaces + cgroups
Guest OS	A full OS per VM	None — shares the host kernel
Typical size	Gigabytes	Megabytes (Pinboard's API: 11 MB)
Start time	Tens of seconds	Milliseconds
Density	Tens per host	Hundreds per host
Best at	Hard isolation, other kernels	Packaging and scaling app processes



It is not either/or: in every cloud, your containers are already running **inside** VMs. Kubernetes nodes are usually VMs too.

WHY IT MATTERS

What containers buy you

One artefact

App, runtime, libraries and config baked into a single image with a checksum.

Environment parity

The bytes that pass your tests are the bytes that run in production.

Fast & dense

Starts in milliseconds, so scaling out and rolling back are cheap operations.

Portable

Laptop, CI runner, on-prem, any cloud — same image, no repackaging.

Immutable

You never patch a running container: you build a new image and replace it.

The unit of orchestration

Everything Kubernetes does from session 05 onwards assumes this unit.

02

What a container really is

No magic, no tiny virtual machine — just a normal Linux process with a restricted view of the world.

THE CORE IDEA

A container is a process

It runs on the host kernel, shows up in the host's process list, and is scheduled like any other process.

- What differs is its **view**: its own PID 1, hostname, network stack and filesystem
- And its **budget**: how much CPU, memory and I/O it may use
- Kill the process and the container is gone

```bash
# on the host
$ ps -e | wc -l
248

$ docker run -it --rm alpine sh
/ # hostname
3f2a9c1b4e77
/ # ps
PID   USER     COMMAND
    1 root     sh
    7 root     ps
```

KERNEL FEATURES

Namespaces, cgroups, overlayfs

Namespaces — what it sees

`pid` · `net` · `mnt` · `uts` · `ipc` · `user` · `cgroup` · `time`. Each one hides the host's version of a resource and gives the process its own.

cgroups v2 — what it may use

Limits and accounts for `cpu`, `memory`, `pids` and `io`. This is what `--memory`, `--cpus` and Kubernetes resource limits actually configure.

overlayfs — what it changes

Stacks read-only image layers under one thin writable layer, so a container starts without copying a filesystem.

All three are plain Linux kernel features — Docker just configures them for you. That is also why Docker Desktop on macOS or Windows quietly runs a small **Linux VM**.

STORAGE VIEW

Layers and the thin writable layer

01
Image layers
Read-only, shared by every container from that image

+

02
Writable layer
Created per container, on top, copy-on-write

=

03
Merged view
What the process sees as `/`

- Editing a file copies it up into the writable layer — the image itself never changes
- Ten containers from one image share the layers on disk and add ten thin layers
- **Delete the container and the writable layer goes with it** — that is why volumes exist (session 03)

LIVE DEMO · 8 MIN

Your first containers — and the isolation, live

`docker run hello-world`, then an interactive `alpine` shell: compare `hostname`, `ps` and `ls /` inside the container with the same commands on the host.

What you'll see · pull → create → start → exit **Images** · hello-world, alpine **Duration** · 8 min

STANDARDS

OCI — why any of this is portable

Image spec

What an image is: a manifest, a config (env, entrypoint, user) and a list of layer tarballs, all addressed by `sha256` digest.

Runtime spec

How to turn an unpacked image plus a JSON config into a running process. `runc` is the reference implementation.

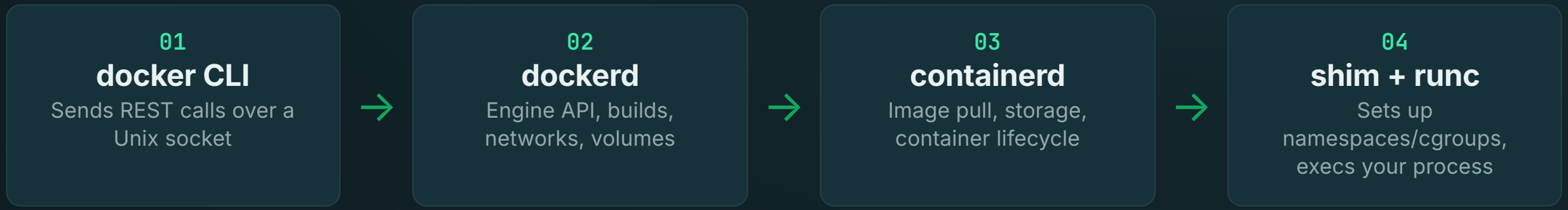
Distribution spec

The HTTP API every registry speaks — Docker Hub, GHCR, ACR, ECR, GAR, Harbor.

✓ Docker is **one implementation** of open standards. An image built by Docker, Podman, Buildah or Kaniko runs anywhere — including on Kubernetes, which does not use Docker at all.

UNDER THE HOOD

The Docker architecture today



- `docker` is only a client — the daemon does the work, which is why remote engines and rootless mode work
- Each container gets its own **shim**, so restarting `dockerd` does not kill running containers
- Kubernetes skips the top two boxes and talks to **containerd via CRI** — same containerd, same runc, same images

03

Images, containers, registries

Three words people use interchangeably and shouldn't — plus how to name an image so you always get the bytes you expect.

VOCABULARY

Image vs. container vs. registry

Image

A read-only, versioned template: filesystem layers plus config. Built once, never changes. `docker image ls`.

Container

A running (or stopped) **instance** of an image, with its own writable layer, name and ID. `docker ps -a`.

Registry

Where images are stored and shared. A **repository** inside it holds the tags of one image, e.g. `library/postgres`.

Class and object, if you write code. One `pinboard-api:1.0` image can back **fifty** running containers, each with its own state.

INSIDE AN IMAGE

What you actually pull

- A **manifest** listing the config and the layers
- A **config blob** — env, entrypoint, user, exposed ports, architecture
- The **layers**, compressed tarballs, pulled in parallel and shared with other images
- Everything named by its `sha256` digest, so identical layers are downloaded once

BASH

```
 $ docker pull postgres:17-alpine
17-alpine: Pulling from library/postgres
9824c27679d3: Pull complete
f9f0b4e0b1cb: Already exists
Digest: sha256:8c9e5f0a...c41b
Status: Downloaded image for postgres:17-alpine

$ docker image inspect postgres:17-alpine \
    --format '{{.Config.Entrypoint}} {{.Os}}/{{.Architecture}}'
[docker-entrypoint.sh] linux/amd64
```

NAMING

Reading an image reference

```
TEXT
   ghcr.io / zonoth / pinboard-api : 1.0 @ sha256:9f2c4e...b7
 ──┬──   ──┬──   ─────┬─────   ─┬─   ────────┬────────
registry  namespace  repository   tag         digest
 (host)    (org/user)   (name)  (movable)  (immutable)
```

YOU TYPE	DOCKER RESOLVES IT TO
`postgres`	`docker.io/library/postgres:latest`
`zonoth/pinboard-api:1.0`	`docker.io/zonoth/pinboard-api:1.0`
`ghcr.io/zonoth/pinboard-api:1.0`	exactly that — a host with a dot means a registry

PIN IT

Tags move. Digests don't.

- A tag is a **mutable pointer**: today's `17-alpine` is not last month's
- `latest` is not "newest" — it is just the default tag name, and nobody has to update it
- A digest is the **content hash**: the same digest is always the same bytes
- Rule of thumb: build with version tags, deploy by digest where it matters

BASH

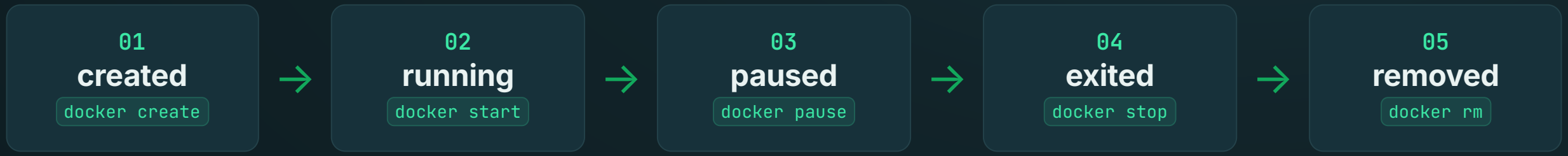
```
# reproducible: exactly these bytes, forever
docker pull postgres@sha256:8c9e5f0a...c41b

# what the tag points at right now
docker image ls --digests postgres
```

⚠ Never deploy `:latest`. Two nodes pulling "the same" tag an hour apart can end up running different code.

STATES

The container lifecycle



- `docker run` = pull (if needed) + create + start, in one command
- An **exited** container still exists: it keeps its logs, its writable layer and its ID until you `rm` it
- A container runs exactly as long as its PID 1 does — when the main process ends, so does the container

LIVE DEMO · 10 MIN

Drive one container through its whole life

Run nginx detached with a published port, read its logs, get a shell inside it, stop it, start it again, and finally remove it — watching `docker ps -a` after every step.

What you'll see · run → logs → exec → stop → start → rm **Image** · nginx:1.28-alpine **Duration** · 10 min

04

The everyday Docker CLI

Twelve flags and six commands cover ninety per cent of what you will type for the rest of the course.

FLAGS

The flags of `docker run`

FLAG	MEANING	EXAMPLE
`-d`	Detached — run in the background, print the ID	`-d nginx:1.28-alpine`
`-it`	Interactive + TTY — for shells and REPLs	`-it alpine sh`
`--rm`	Delete the container when it exits	`--rm alpine date`
`--name`	A stable name instead of `quirky_tesla`	`--name pinboard-db`
`-p`	Publish **host:container** port	`-p 5432:5432`
`-e`	Set an environment variable	`-e POSTGRES_PASSWORD=pinboard`

Order matters: `docker run [flags] IMAGE [command]`. Anything after the image name is passed to the container, not to Docker.

OBSERVE

Looking inside a running container

```bash
 docker ps                              # what is running (add -a for everything)
docker logs -f --tail 50 pinboard-db  # stdout/stderr, follow like tail -f
docker exec -it pinboard-db psql -U pinboard   # a second process inside it
docker inspect pinboard-db            # the full JSON truth: mounts, env, network

# --format turns that JSON into one useful line
docker inspect pinboard-db \
  --format '{{.State.Status}} {{.NetworkSettings.IPAddress}}'
docker inspect -f '{{range .Config.Env}}{{println .}}{{end}}' pinboard-db
```

- Containers log to **stdout/stderr** — never to a file inside the container
- `exec` starts an **extra** process; it does not restart the container, and it dies when you exit

MAINTAIN

Measuring and cleaning up

- `docker stats` — live CPU, memory and I/O per container, straight from cgroups
- `docker system df` — where your disk went: images, containers, volumes, build cache
- Prune early and often; a laptop running this course will fill up otherwise

⚠ `prune -a --volumes` also deletes **data volumes**. Read the prompt before you type `y`.

BASH

```
 docker stats --no-stream
docker system df -v

# safe: dangling images, stopped
# containers, unused networks
docker system prune

# aggressive: every unused image
docker system prune -a
```

WHERE IMAGES LIVE

Registries you will meet

Docker Hub

The default. Official images live in `library/`. Anonymous pulls are rate-limited — `docker login` raises the limit.

GHCR

`ghcr.io/<org>/<repo>` — sits next to your source, authenticates with a GitHub token. What we use for the course images.

Cloud registries

Azure ACR, AWS ECR, Google Artifact Registry — IAM-integrated, private by default, close to your cluster.

Self-hosted

Harbor or the plain `registry:2` image when everything must stay inside the network.

LIVE DEMO · 6 MIN

Inspect, measure, prune

Pull an image by digest, pick two fields out of `docker inspect` with `--format`, watch `docker stats` while nginx serves requests, then reclaim the disk.

What you'll see · digest pull · --format · stats · system df **Images** · nginx, alpine **Duration** · 6 min

LAB 01 · 30–40 MIN

Run the Pinboard database tier

Pinboard needs somewhere to keep its notes. You will run `postgres:17-alpine` as a container, create the `notes` table by hand, and discover — the hard way — what happens to that data when the container is removed.

You'll practise: `run -d -e -p --name` · `exec` into `psql` · `logs` · `inspect --format` · `stop` / `start` / `rm` · `system prune`

Key takeaways

✓ A container is a process

Namespaces limit what it sees, cgroups what it uses, overlayfs what it changes. No guest OS, no magic.

✓ Image ≠ container

Images are immutable templates in a registry; containers are instances with a throwaway writable layer.

✓ Name images precisely

`registry/namespace/repo:tag@digest`. Tags move, digests don't, and `latest` means nothing.

✓ Next up

Session 02 — building images: Dockerfiles, BuildKit, the cache, and Pinboard's 11 MB multi-stage image.

Zonoth

Questions?

Up next — Session 02: Building images.
We stop running other people's
containers and start building our own.